

Contents

07 — Plano de Ação: Comandos Específicos por Material	1
1. Contexto e objetivo	1
2. Estado atual e lacunas	2
3. Decisões de design	2
4. Fases de implementação	3
5. Riscos e mitigações	5
6. Critérios de aceite (definition of done)	5
7. Contrato de saída	6
8. Resultado da implementação (evidência)	6

07 — Plano de Ação: Comandos Específicos por Material

Status: concluído (2026-08-05 — implementado e validado com smoke test em kit-inlego)

Data: 2026-08-05 **Escopo:** sistema de comandos da Fábrica de Materiais de Comunicação (SPEC_COMANDOS.md, .claude/commands/*.md, docs de superfície) **Impacto em scripts Python:** nenhum (a infraestrutura de --apenas do auditar-projeto.py já suporta os 9 tipos de material — verificado)

1. Contexto e objetivo

Hoje o operador consegue regenerar pontualmente 3 dos 9 materiais com comandos específicos (/gerar-pdf, /gerar-landing, /gerar-apresentacao). As 3 **variantes de arte** são cobertas por um único comando com flag (/gerar-arte <slug> [--tamanho ...]), e 3 **materiais não têm comando nenhum**: Textos de Apoio, Kit do Consultor e Kit do Distribuidor.

Objetivo deste plano: dar a cada um dos 9 materiais um **comando específico**, mantendo o princípio arquitetural de SPEC_COMANDOS.md — o procedimento canônico vive em um único lugar (fonte de verdade) e os arquivos em .claude/commands/*.md são apenas ponteiros finos de descoberta (lição do bug histórico de TIPOS_VALIDOS duplicado).

2. Estado atual e lacunas

#	Material (tipo interno)	Comando (situação final)	Situação
1	PDF / apostila (pdf)	/gerar-pdf <slug>	Existe (inalterado)
2	Landing Page (landing-page)	/gerar-landing <slug>	Existe (inalterado)
3	Apresentação (apresentacao)	/gerar-apresentacao <slug>	Existe (inalterado)
4	Arte 1080×1080 (arte-01)	/gerar-arte-1080x1080 <slug>	Criado
5	Arte 1080×1350 (arte-02)	/gerar-arte-1080x1350 <slug>	Criado
6	Arte 1080×1920 (arte-03)	/gerar-arte-1080x1920 <slug>	Criado
7	Textos de Apoio (textos)	/gerar-textos <slug>	Criado
8	Kit do Consultor (kit-consultor)	/gerar-kit-consultor <slug>	Criado
9	Kit Distribuidor (kit-distribuidor)	/gerar-kit-distribuidor <slug>	Criado

/gerar-arte <slug> [--tamanho ...] foi mantido como guarda-chuva retrocompatível (regenera as 3 variantes, delegando cada uma ao comando específico).

/esbocar (entrevista) e /produzir-comunicacao-completa <slug> (lote completo autônomo) permanecem inalterados — são o Passo 1 e o Passo 2 do fluxo, não materiais.

3. Decisões de design

1. **Nomenclatura por dimensão nas artes:** /gerar-arte-1080x1080, /gerar-arte-1080x1350, /gerar-arte-1080x1920. A dimensão é a linguagem que o operador já usa no /esbocar (Passo 4, Parte 2/3). O mapeamento para o tipo interno (arte-01|02|03) fica documentado apenas no SPEC_COMANDOS.md, como já acontece com --tamanho hoje.
2. **/gerar-arte continua existindo como comando “guarda-chuva”** (sem --tamanho regenera as 3 variantes) — retrocompatível e útil para regenerar tudo de arte de uma vez. Sua seção passa a apontar os 3 comandos específicos.

3. **Kits com CTA próprio:** /gerar-kit-consultor e /gerar-kit-distribuidor são comandos distintos, mas **compartilham a mesma copy** (kits/copies.json, 10 copies — ver SPEC_KITS.md). A primeira execução garante o copies.json via redator-kit-copy inline; a segunda reaproveita sem regravar. Mesma disciplina do Passo 2.7 do /produzir-comunicacao-completa.
4. **Artes com copy compartilhada:** idem para arte/copies.json (3 copies) — mesma disciplina do Passo 2.5. Formato (dimensão) e copy são eixos ortogonais.
5. **Padrão procedural idêntico ao dos comandos existentes:** pré-condição de brief_criativo.json → garantir tipo em materiais_selecionados → garantir copies compartilhadas (se aplicável) → despachar subagente-produtor → subagente-revisor-marca só para o tipo → auditar-projeto.py --estrito --apenas <tipo> → empacotar-projeto.py → relatório telegráfico.
6. **Zero mudança em scripts:** auditar-projeto.py --apenas já aceita os 9 tipos (TIPOS_VALIDOS completo, linha 25), e pool-materiais.py/empacotar-projeto.py já conhecem textos, kit-consultor e kit-distribuidor. O trabalho é 100% de documentação de orquestração + ponteiros.

4. Fases de implementação

Fase 1 — SPEC_COMANDOS.md (fonte única de verdade)

Arquivo: SPEC_COMANDOS.md

1. Atualizar a lista de comandos da introdução (linhas 4–5) para incluir os 6 novos.
2. Adicionar as seções ## /gerar-textos, ## /gerar-kit-consultor, ## /gerar-kit-distribuidor, ## /gerar-arte-1080x1080, ## /gerar-arte-1080x1350, ## /gerar-arte-1080x1920, seguindo o padrão das existentes:
 - Pré-condição: output/<slug>/brief_criativo.json existe; senão, parar com a mensagem canônica “Rode /esbocar (ou /produzir-comunicacao-completa <slug>) primeiro”.
 - Se o tipo não estiver em config_projeto.materiais_selecionados, adicioná-lo.
 - Artes: garantir arte/copies.json (3 copies) via redator-arte inline se ausente; senão reaproveitar. Despachar 1 subagente-produtor-arte para a variante pedida.
 - Kits: garantir kits/copies.json (10 copies) via redator-kit-copy inline se ausente; senão reaproveitar. Despachar 1 subagente-produtor-kit para a variante pedida (CTA de brand/kits-conexao.json resolvido pelo compilador).
 - Textos: despachar subagente-produtor-textos.
 - Revisão: 1 subagente-revisor-marca só para o tipo da rodada.
 - Auditoria: python scripts/auditar-projeto.py <slug> --estrito --apenas <tipo>.
 - Empacotamento: python scripts/empacotar-projeto.py <slug>.
 - Relatório telegráfico (REGRA 2): paths dos artefatos, decisões de design, faltantes, sugestões de legenda/CTA.
3. Ajustar a seção ## /gerar-arte para ser o guarda-chuva: resolver variantes via --tamanho (ou as 3) e **delegar a execução de cada variante ao procedimento do comando específico correspondente** (sem duplicar o passo a passo).

Verificação: conferência de que cada um dos 9 materiais possui seção própria no arquivo (grep "`^## /gerar`" `SPEC_COMANDOS.md` retorna 10 seções: esboçar + produzir-comunicacao-completa + 8 pontuais) e que nenhum trecho de procedimento está duplicado entre seções.

Fase 2 — Ponteiros finos em `.claude/commands/`

Arquivos novos (6):

- `.claude/commands/gerar-textos.md`
- `.claude/commands/gerar-kit-consultor.md`
- `.claude/commands/gerar-kit-distribuidor.md`
- `.claude/commands/gerar-arte-1080x1080.md`
- `.claude/commands/gerar-arte-1080x1350.md`
- `.claude/commands/gerar-arte-1080x1920.md`

Cada um com frontmatter description (uma linha, estilo dos existentes), título e a instrução canônica: "Leia `SPEC_COMANDOS.md` por completo agora e execute exatamente o que está escrito na seção `/gerar-XXX`. `$ARGUMENTS` = `<slug>` referenciado naquele documento." **Nunca** copiar o procedimento para dentro do ponteiro (bug histórico).

Atualizar `.claude/commands/gerar-arte.md` apenas se a assinatura mudar (não muda — continua `--tamanho` opcional).

Verificação: 1 arquivo por comando, 9 ponteiros no total, todos referenciando `SPEC_COMANDOS.md`; description no frontmatter de todos (requisito do Claude Code).

Fase 3 — Docs de superfície (listas de comandos)

Arquivos: `AGENTS.md` (linhas 22–23), `CLAUDE.md` (linhas 19–22), `CODEBUDDY.md` (linhas 43–44), `QODER.md` (linhas 43–44), `GEMINI.md` (linhas 43–44).

Atualizar a lista/descrição de comandos para incluir os 6 novos, mantendo o formato existente de cada arquivo.

Verificação: `grep -c "gerar-" AGENTS.md CLAUDE.md CODEBUDDY.md QODER.md GEMINI.md` retorna a contagem esperada de comandos por arquivo (12 no `AGENTS.md` com `gerar-`, etc.) e nenhuma lista menciona comando que não exista em `SPEC_COMANDOS.md` (conferir cruzado com `grep "^## /" SPEC_COMANDOS.md`).

Fase 4 — Validação funcional (smoke test em projeto real)

1. `python scripts/verificar-consistencia-pipeline.py --estrito` — sem mudança de materiais este teste não deve quebrar; serve como smoke de sanidade.
2. Com um slug real já esboçado (ex.: `kit-inlego` se existir em `output/`):
 - rodar `/gerar-textos <slug>` e conferir `exit 0` do `auditar-projeto.py --apenas textos + arquivos .txt` em `output/<slug>/textos/`;
 - rodar `/gerar-arte-1080x1350 <slug>` e conferir 3 PNGs 1080×1350 novos em `output/<slug>/arte-02/` (`validar-dimensoes.py`);
 - rodar `/gerar-kit-consultor <slug>` e conferir 10 PNGs 1080×1350 + `texto_whatsapp.txt` por item (`validar-kit.py`);

- rodar `/gerar-kit-distribuidor <slug>` e conferir que **não** regerou `copies.json` (reuso, não regravação) e que o CTA/assinatura é o de distribuidor.
3. Conferir que `manifesto_materiais.json` reflete os materiais regenerados após cada comando (exit 0 do `empacotar-projeto.py`).

Verificação: exit codes 0 em todas as validações; nenhum comando parou pedindo pergunta ao operador (REGRA 3).

Fase 5 — Fechamento e relatório

1. Revisar este plano e marcar como concluído (trocar “proposto” por “concluído” no cabeçalho, se implementado).
2. Atualizar a tabela da seção 2 com a situação final.
3. Atualizar `docs/06-plano-expansao-kits-consultor-distribuidor.md` (linhas 229–230), que antecipava “um comando pontual equivalente a `/gerar-arte`” para kits — agora existe e deve ser referenciado.

Verificação: nenhuma referência a “comando futuro/inexistente” restante em `docs/` (`grep -rn "comando pontual equivalente" docs/` vazio) e versões `.md/` `.pdf` deste plano em `docs/`.

5. Riscos e mitigações

Risco	Mitigação
Divergência entre <code>SPEC_COMANDOS.md</code> e ponteiros (bug histórico <code>TIP0S_VALID0S</code>)	Ponteiros continuam finos; canônico só no <code>SPEC</code> ; verificação da Fase 2
Race de <code>copies.json</code> se os 2 kits (ou artes) forem gerados em paralelo antes da 1ª garantia	Garantia inline de <code>kits/copies.json/arte/copies.json</code> antes de qualquer fan-out (disciplina dos Passos 2.5/2.7)
Esquecer doc de superfície (<code>AGENTS.md</code> , <code>CLAUDE.md</code> , <code>CODEBUDDY.md</code> , <code>QODER.md</code> , <code>GEMINI.md</code>)	Fase 3 dedicada com verificação por <code>grep</code>
Comando novo com <code>--apenas</code> que aceite tipo errado	<code>TIP0S_VALID0S</code> já cobre os 9; smoke test da Fase 4 válida na prática
<code>auditar-projeto.py --apenas</code> sem o material marcado em <code>materiais_selecionados</code>	Cada comando adiciona o tipo ao <code>materiais_selecionados</code> antes de auditar (padrão já existente)

6. Critérios de aceite (definition of done)

1. Os **9 materiais** têm comando específico documentado em `SPEC_COMANDOS.md` (seção própria), e `/gerar-arte` segue como guarda-chuva retrocompatível.
2. **6 ponteiros novos** em `.claude/commands/`, todos com `frontmatter description` e nenhum duplicando procedimento.
3. **5 docs de superfície** atualizados (`AGENTS.md`, `CLAUDE.md`, `CODEBUDDY.md`, `QODER.md`, `GEMINI.md`).

4. **Smoke test** da Fase 4 passa: /gerar-textos, /gerar-arte-1080x1350, /gerar-kit-consultor e /gerar-kit-distribuidor em um slug real, com exit 0 nas validações, reuso de copies.json confirmado e manifesto atualizado.
5. Nenhuma mudança em scripts/*.py (a infra já suporta os 9 tipos) — se alguma for necessária durante o smoke test, é surpresa e deve ser reportada antes de prosseguir.

7. Contrato de saída

Ao concluir a implementação, o relatório deve entregar: (a) comandos criados e a seção de cada um no SPEC_COMANDOS.md, (b) evidência dos exit codes dos validadores no smoke test, (c) qualquer surpresa encontrada (ex.: validador que falha para textos em projeto real), (d) sugestões de atalho de uso para o operador.

8. Resultado da implementação (evidência)

Fases 1–3 (docs): concluídas — SPEC_COMANDOS.md com 12 seções de comando (esbocar + produzir-comunicacao-completa + 10 pontuais), 12 ponteiros em .claude/commands/ (todos com description), e os 5 docs de superfície (AGENTS.md, CLAUDE.md, CODEBUDDY.md, QODER.md, GEMINI.md) com a lista atualizada.

Fase 4 (smoke test em kit-inlego, todos os exit codes observados):

Comando	Validação	Resultado
/gerar-textos kit-inlego	validar-textos.py + auditar-projeto.py -- apenas textos	CONFORME (3 .txt UTF-8)
/gerar-kit-consultor kit-inlego	validar-kit.py + auditar-projeto.py --apenas kit-consultor	CONFORME (10 PNGs 1080×1350)
/gerar-kit-distribuidor kit-inlego	validar-kit.py + auditar-projeto.py --apenas kit-distribuidor	CONFORME (CTA/assinatura de distribuidor)
/gerar-arte-1080x1350 kit-inlego	validar-dimensoes.py + auditar-projeto.py -- apenas arte-02	CONFORME (3 PNGs 1080×1350)

- **Reuso de copies confirmado:** kits/copies.json e arte/copies.json não foram regravados (mtime inalterado) — os comandos reaproveitaram as copies existentes, sem nova chamada de LLM.
- **Kits divergem só no CTA:** conteudo.json idêntico exceto cta (“Fale com seu consultor Conexão” × “Peça ao seu distribuidor Conexão”).
- **Empacotamento final:** manifesto_materiais.json 9/9 entregues após a última regeneração.
- **Surpresa pré-existente (fora do escopo):** o material pdf de kit-inlego e kit-master-flex já estava NAO CONFORME antes desta implementação — falha técnica de capa (validar-pdf.py: “paragrafo da capa com 1 linha(s)”, “paragrafo da capa fora do bloco”, rótulo genérico “Guia de

Treinamento”). Nenhum script foi alterado neste plano; a correção é tema de um plano próprio (template template_apostila.typ
 ▶ validar-pdf.py).

Anexo A — Mapeamento de comandos (implementado)

Comando novo	Tipo interno	Subagente	Auditoria	Copies compar- tilhadas
/gerar-textos <slug>	textos	subagente- produtor- textos	--apenas textos	—
/gerar-kit- consultor <slug>	kit-consultor	subagente- produtor-kit	--apenas kit- consultor	kits/ copies.json
/gerar-kit- distribuidor <slug>	kit- distribuidor	subagente- produtor-kit	--apenas kit- distribuidor	kits/ copies.json
/gerar- arte-1080x1080 <slug>	arte-01	subagente- produtor-arte	--apenas arte-01	arte/ copies.json
/gerar- arte-1080x1350 <slug>	arte-02	subagente- produtor-arte	--apenas arte-02	arte/ copies.json
/gerar- arte-1080x1920 <slug>	arte-03	subagente- produtor-arte	--apenas arte-03	arte/ copies.json